Data Communications

Lecture #8

What we have discussed

- Transport layer services
- Multiplexing / Demultiplexing
- Connectionless transport: UDP

Today…

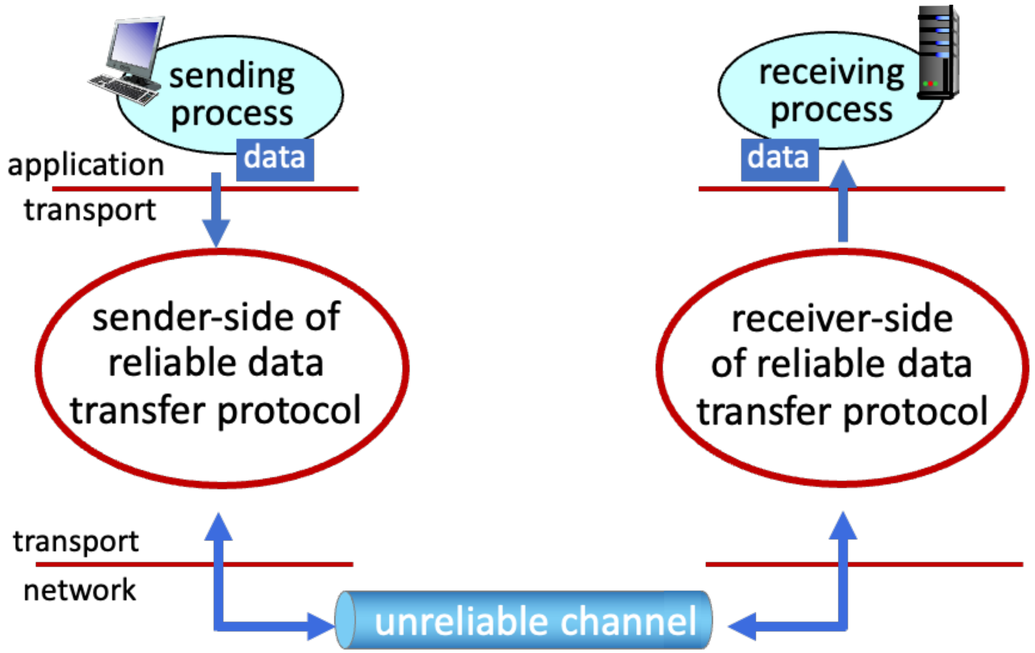
- Principles of reliable data transfer

Principle of reliable data transfer

Reliable service abstraction

Principle of reliable data transfer

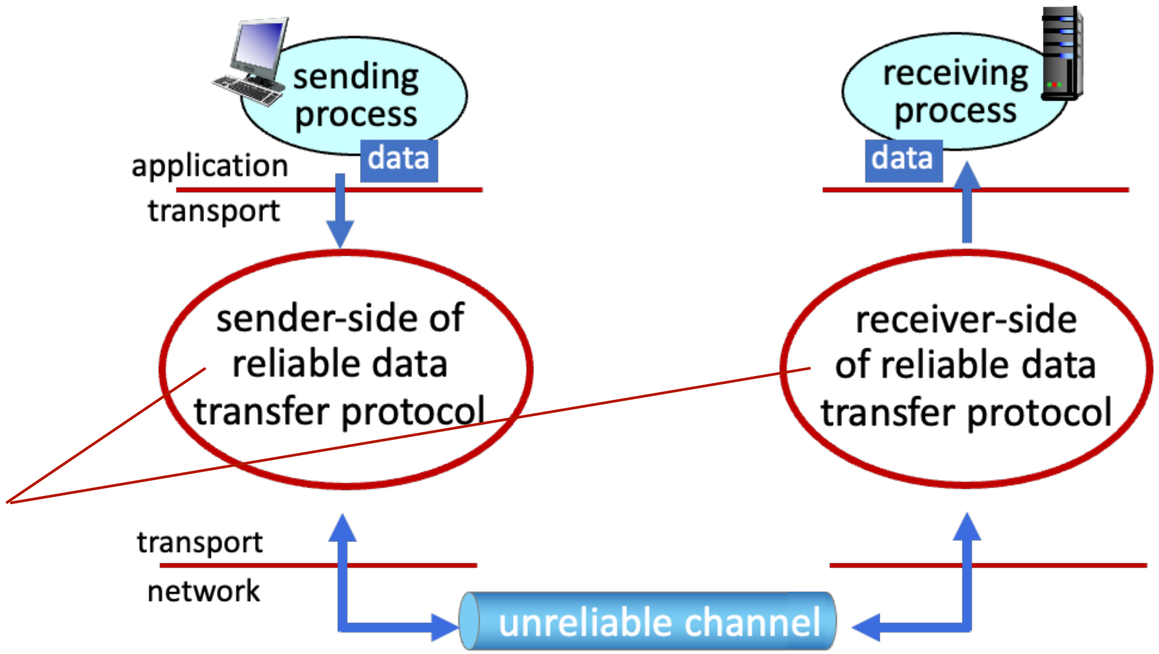
Reliable service abstraction



Reliable service implementation

Principle of reliable data transfer

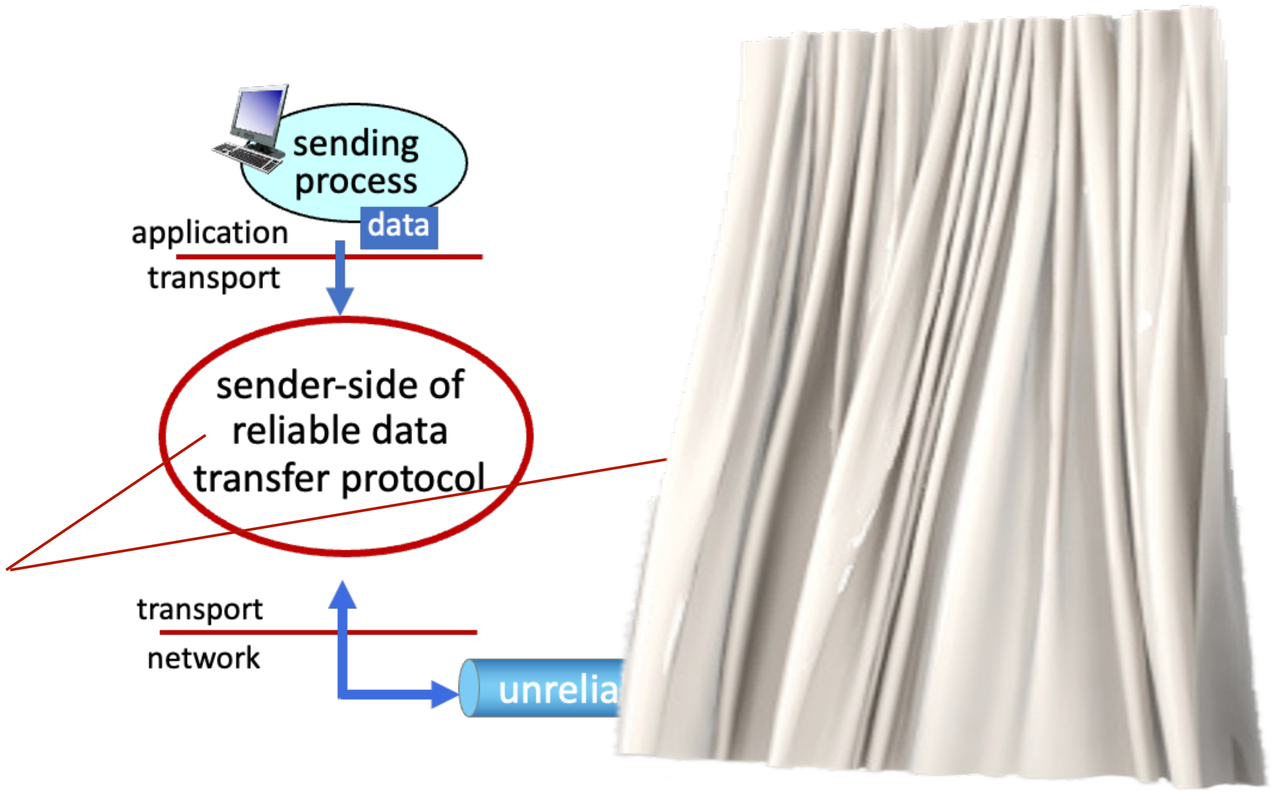
Complexity of reliable data transfer protocol will depend (strongly) on characteristics of unreliable channel (lose, corrupt, reorder data?)



Reliable service implementation

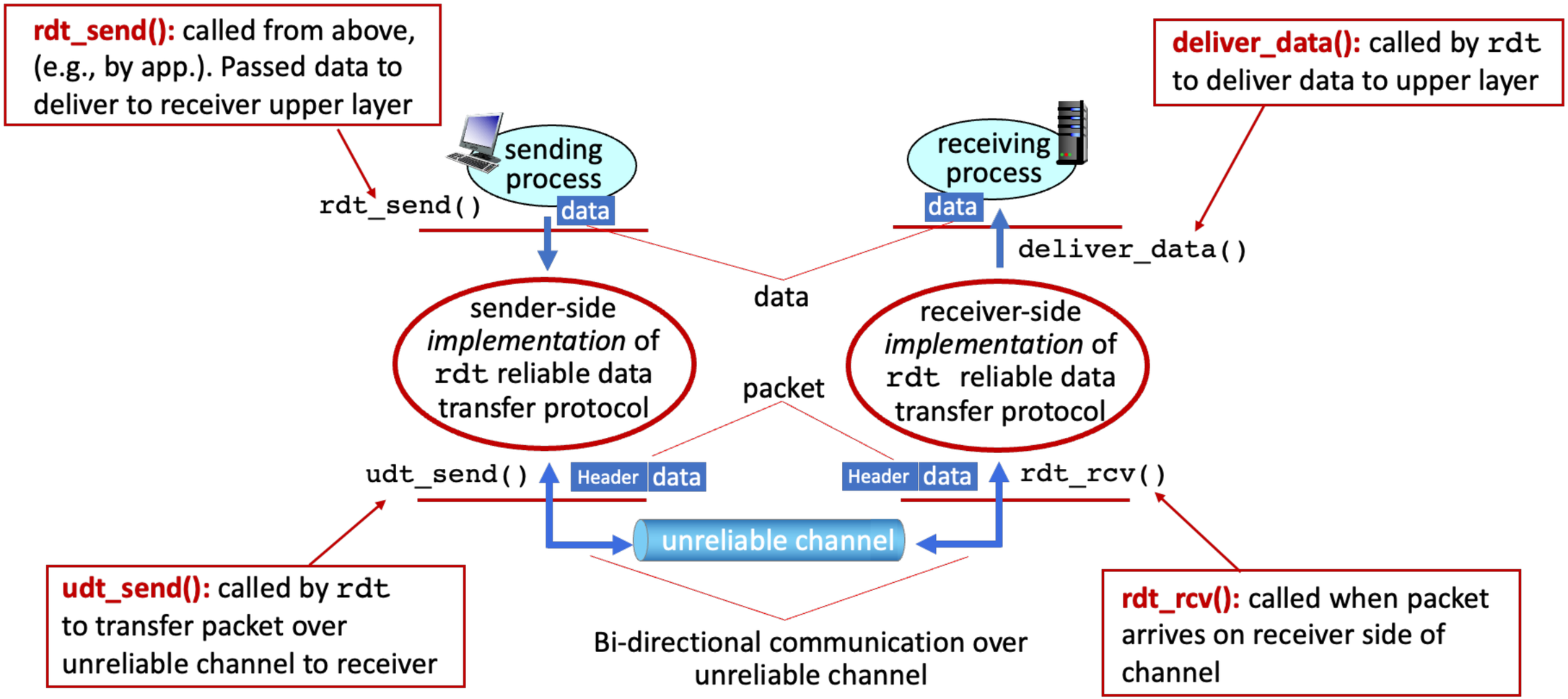
Principle of reliable data transfer

Sender, receiver do not know the "state" of each other (e.g., was a message received?) unless communicated via a message



Reliable service implementation

Reliable data transfer protocol (rat): interfaces

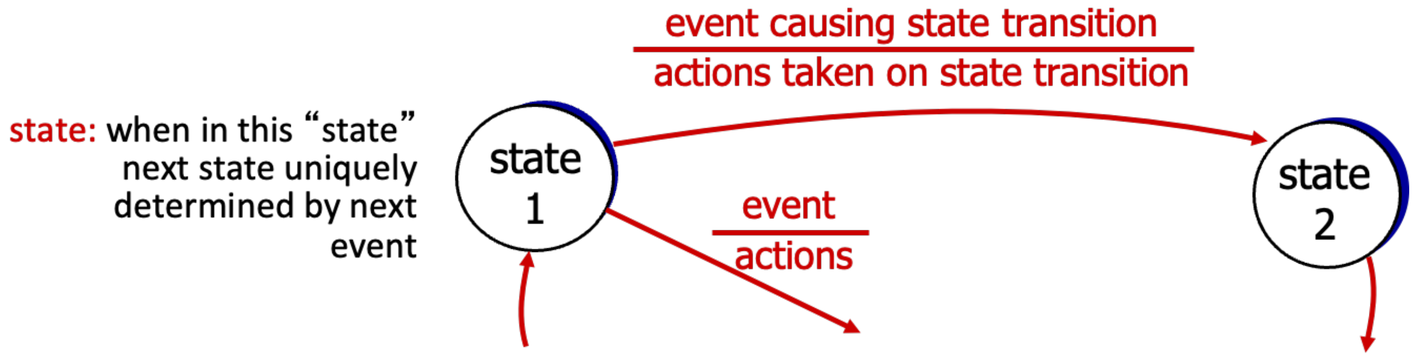


Reliable data transfer: getting started

- We will incrementally develop sender, receiver sides of reliable data transfer protocol (rdt)

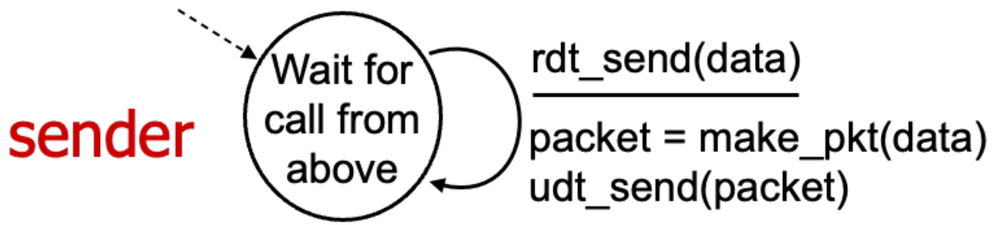
- Consider only unidirectional data transfer
  - But control info will flow in both directions!

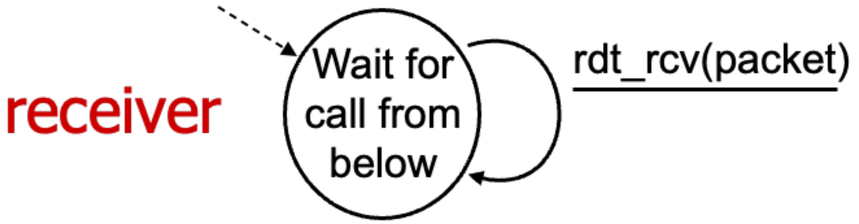
- Use finite state machine (FSM) to specify sender and receiver



rdt 1.0: reliable transfer over a reliable channel

- Underlying channel perfectly reliable
  - No bit errors
  - No loss of packets
- Separate FSMs for sender, receiver
  - Sender sends data into underlying channel
  - Receiver reads data from underlying channel





rdt 2.0: reliable with bit errors

- Underlying channel may flip bits in packet
  - Checksum (e.g., Internet checksum) can be employed to **detect** bit errors

- Q) How to recover from errors?

How do humans recover from “errors” during conversation?

rdt 2.0: reliable with bit errors

- Underlying channel may flip bits in packet
  - Checksum (e.g., Internet checksum) can be employed to **detect** bit errors

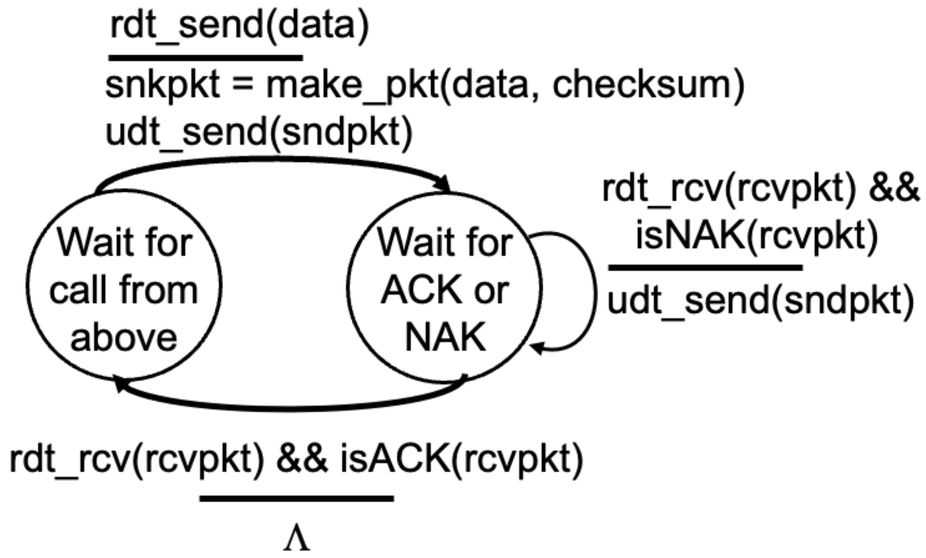
- Q) How to recover from errors?
  - Acknowledgement (ACKs): receiver explicitly tells sender that packet received is OK
  - Negative acknowledgement (NAKs): receiver explicitly tells sender that packet received had errors
  - Sender retransmits packet on receipt of NAK

Stop and wait

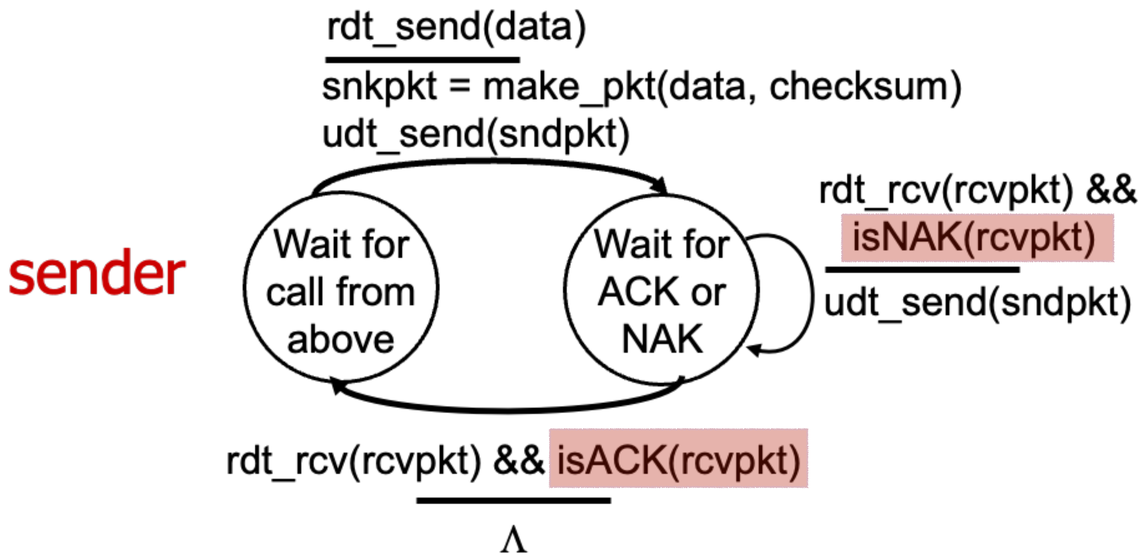
Sender sends one packet, then waits for receiver response

rdt 2.0: FSM specifications

sender



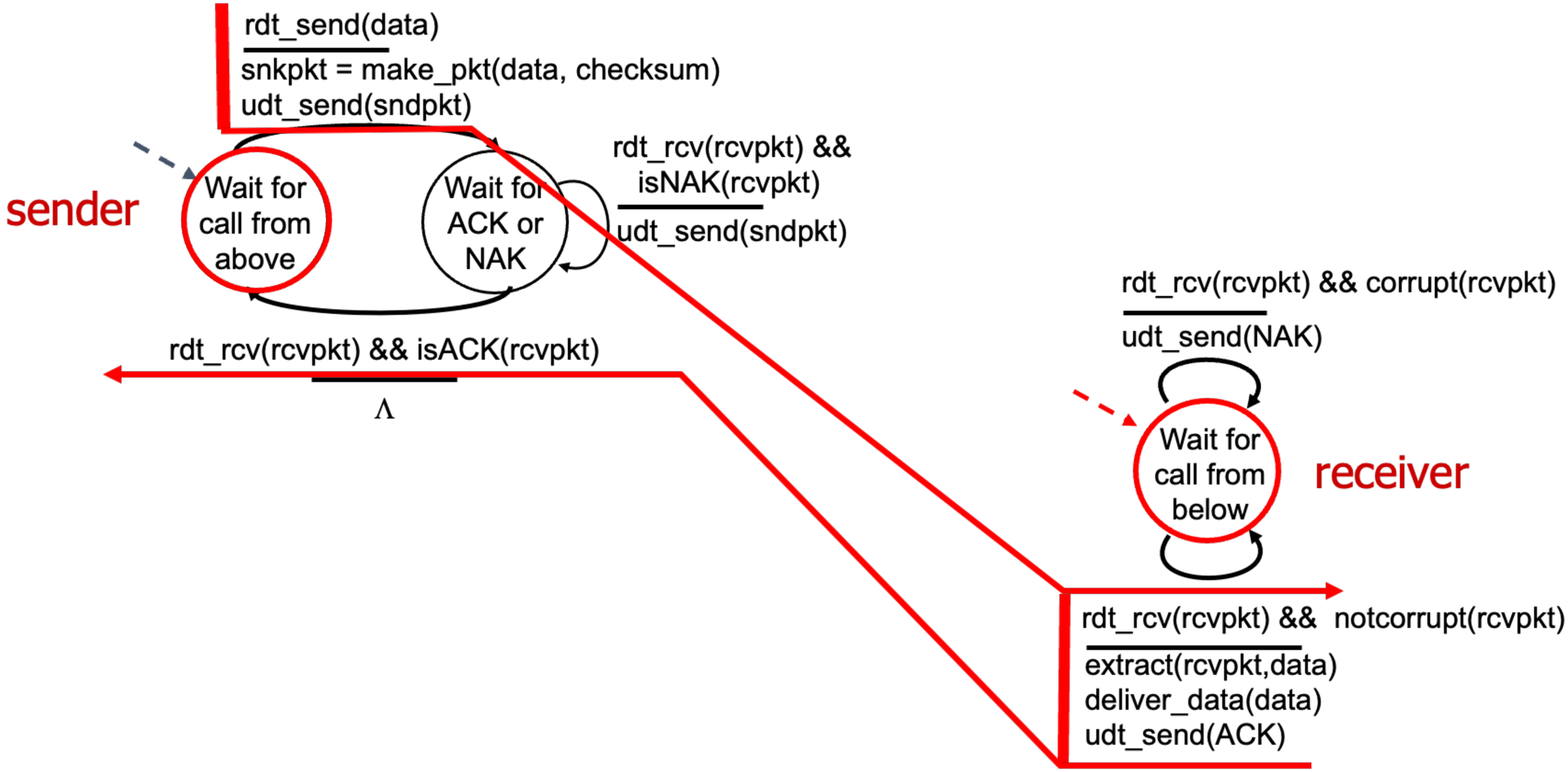
rdt 2.0: FSM specifications



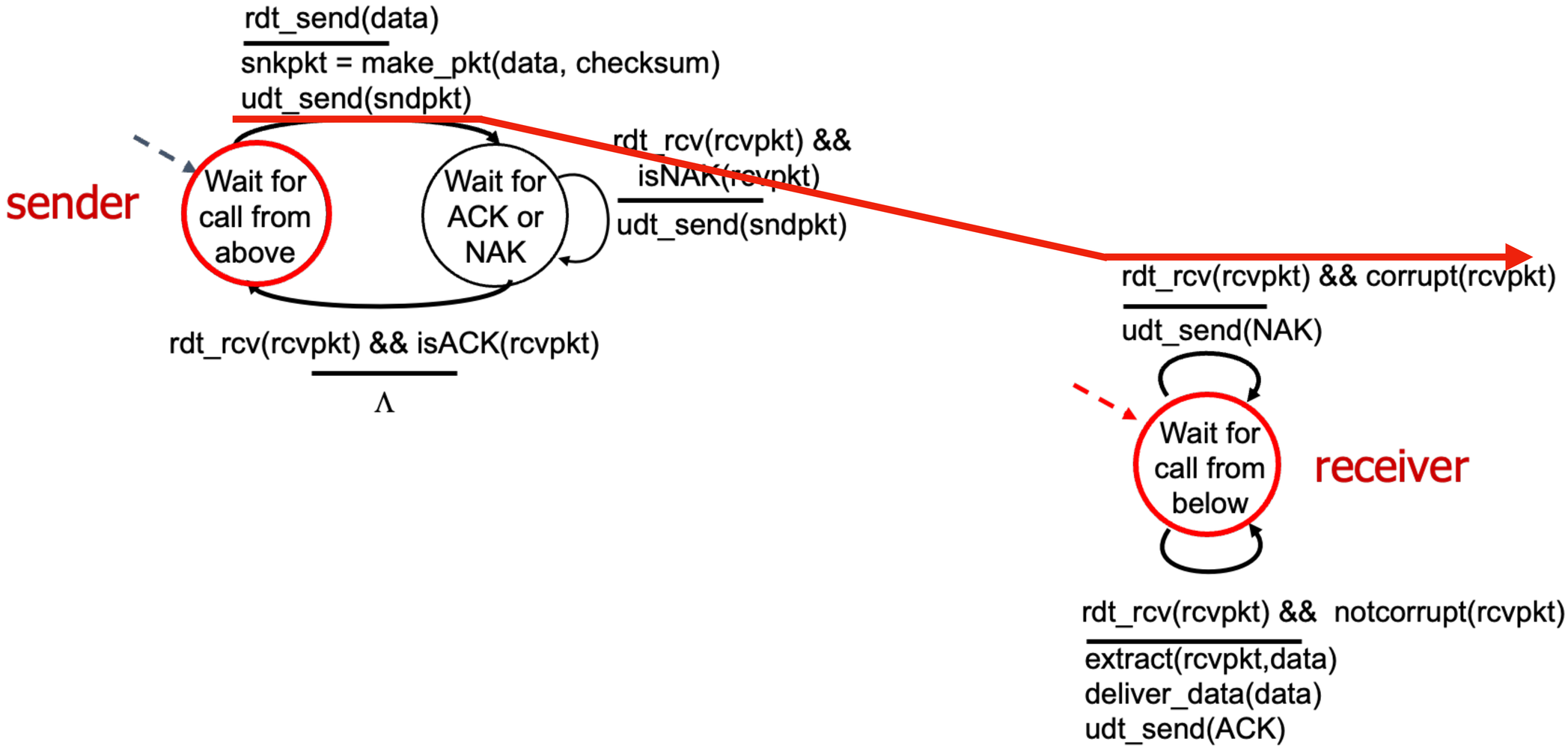
Note: “state” of receiver (did the receiver get my message correctly?) isn’t known to sender unless somehow communicated from receiver to sender

→ that’s why we need a protocol

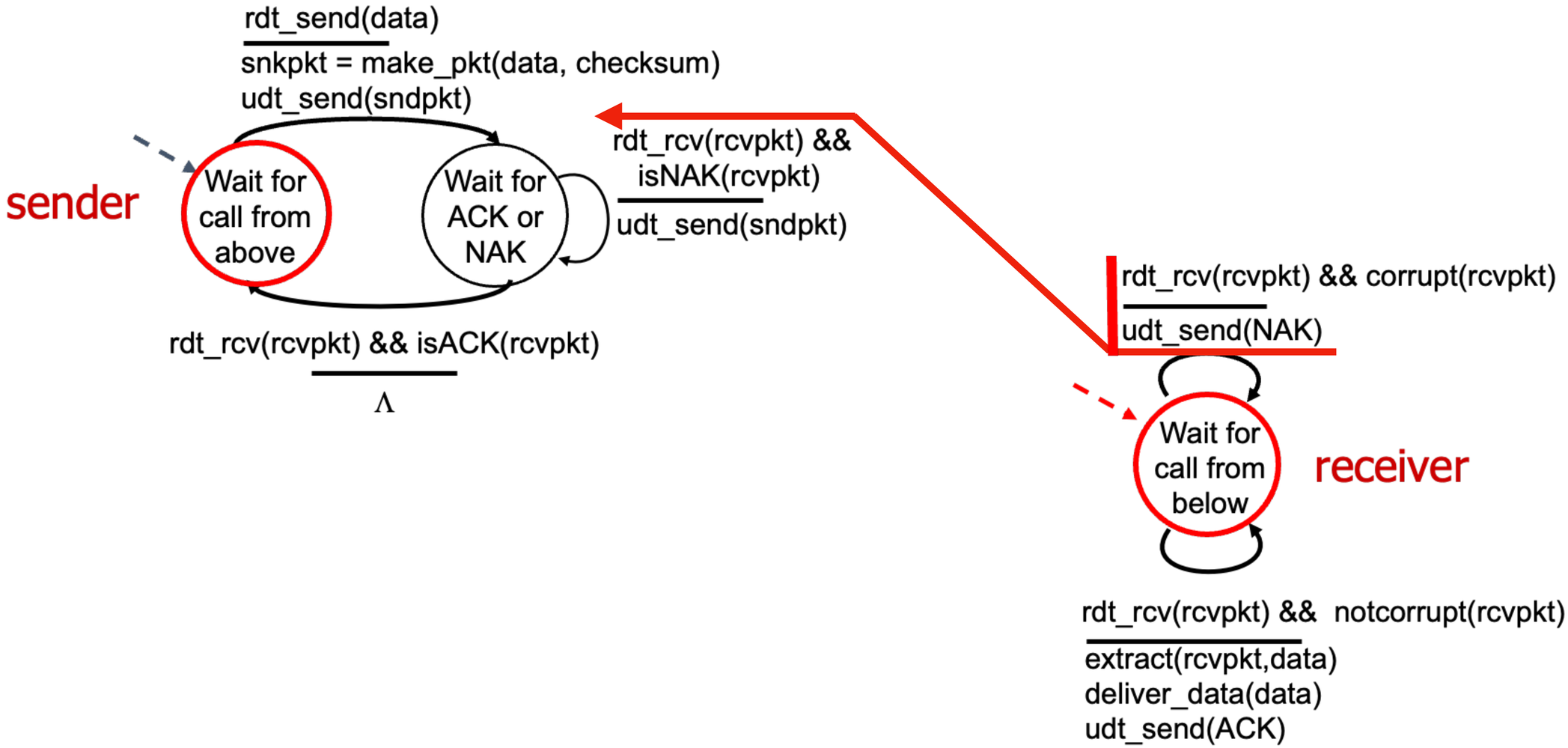
rdt 2.0: operation with no errors



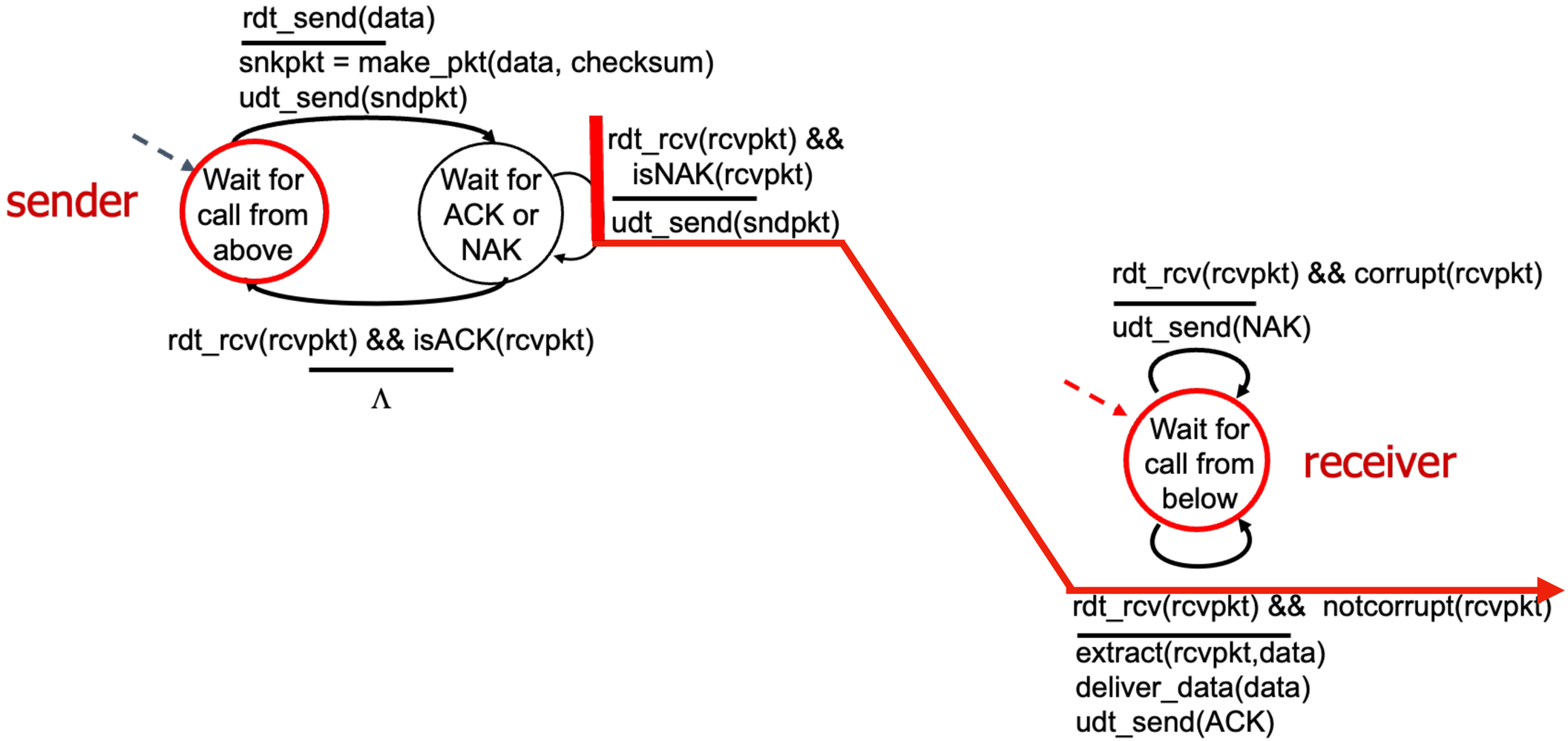
rdt 2.0: corrupted packet scenario



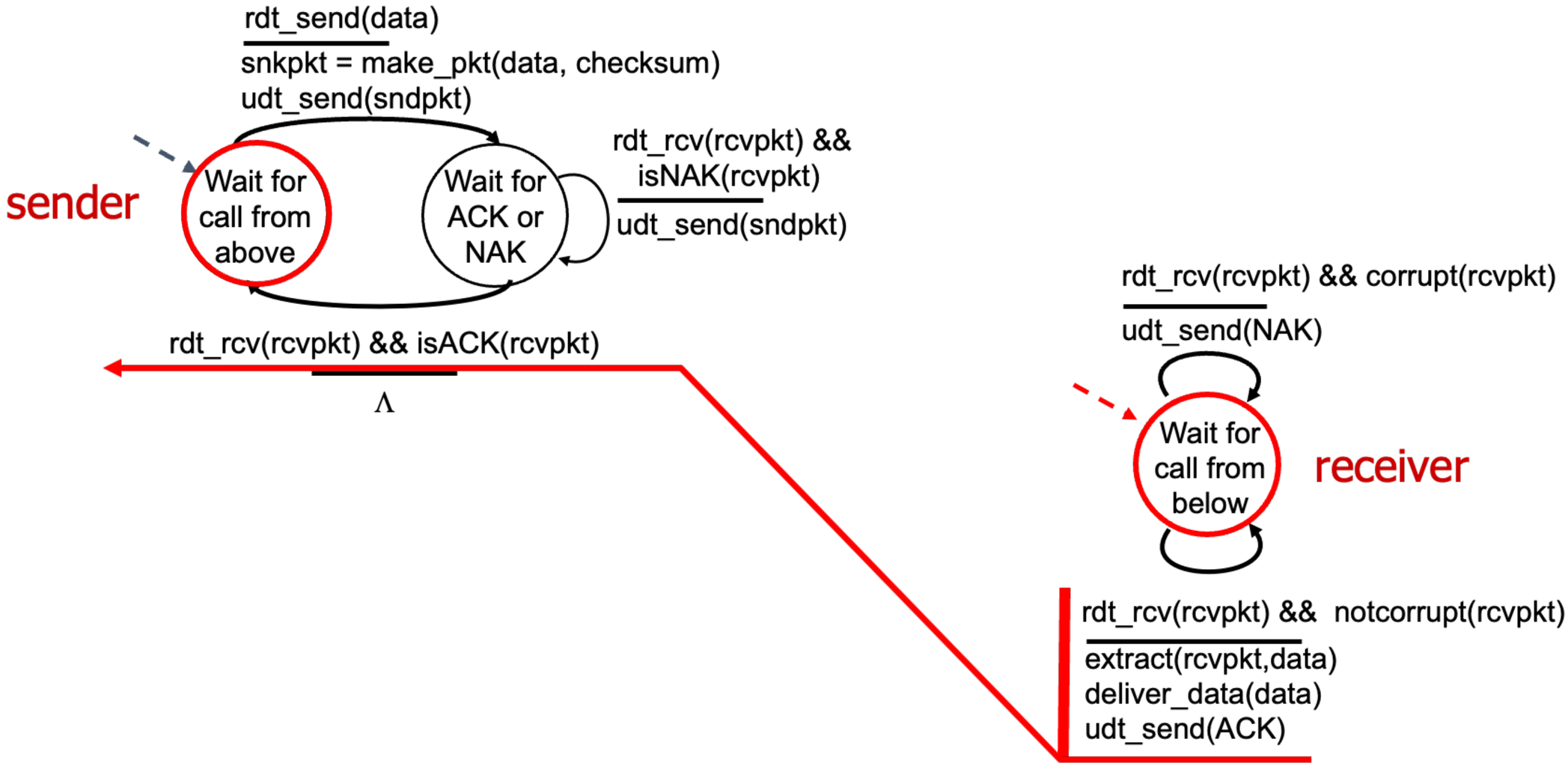
rdt 2.0: corrupted packet scenario



rdt 2.0: corrupted packet scenario



rdt 2.0: corrupted packet scenario



rdt 2.0 has a fatal flaw!

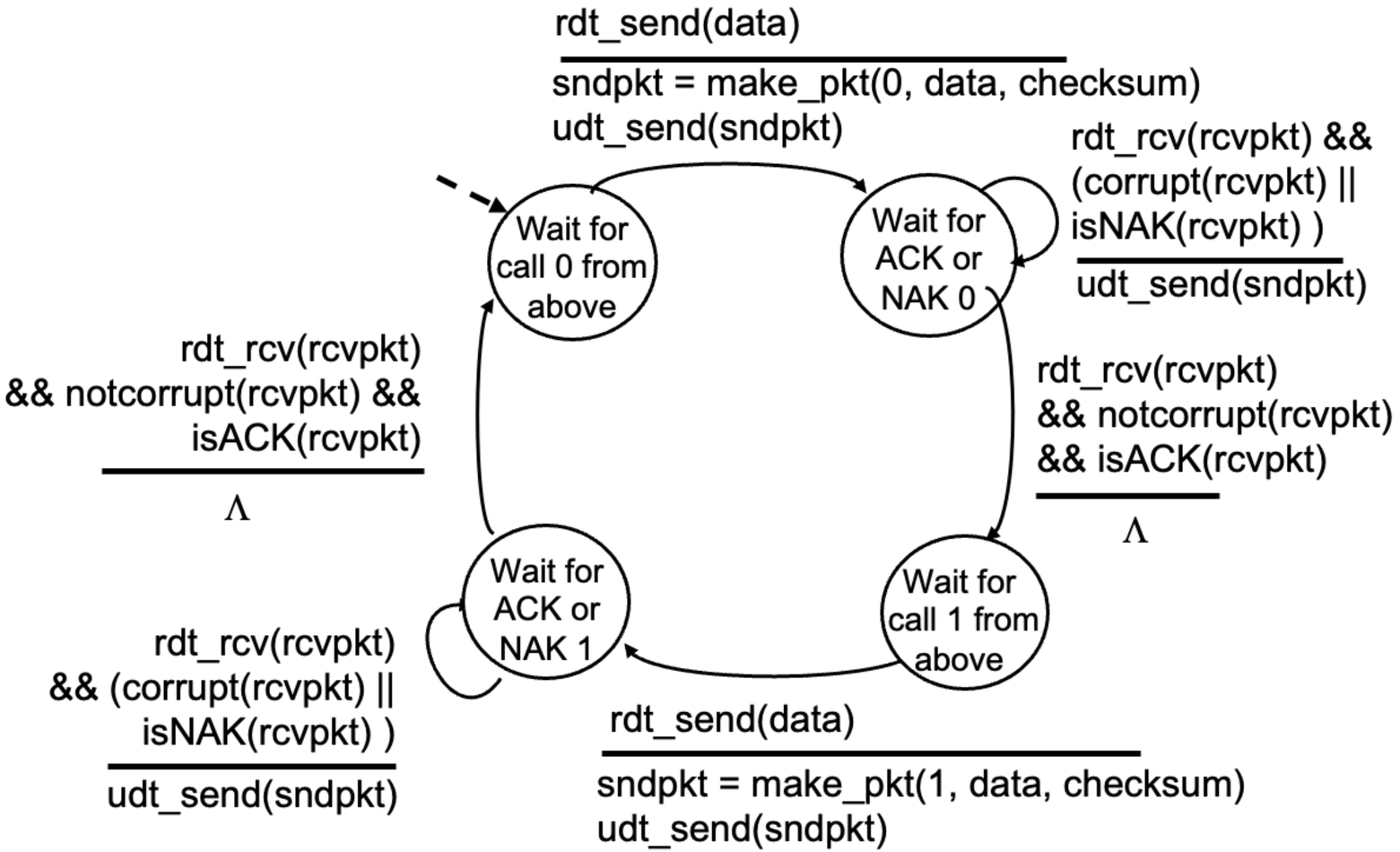
- What happens if ACK/NAK is corrupted?
  - Sender does not know what happened at receiver
  - Can’t just retransmission: possible duplicate

- Handling duplicates
  - Sender retransmits current packet if ACK/NAK corrupted
  - Sender adds sequence number to each packet
  - Receiver discards (doesn’t deliver up) duplicate packets

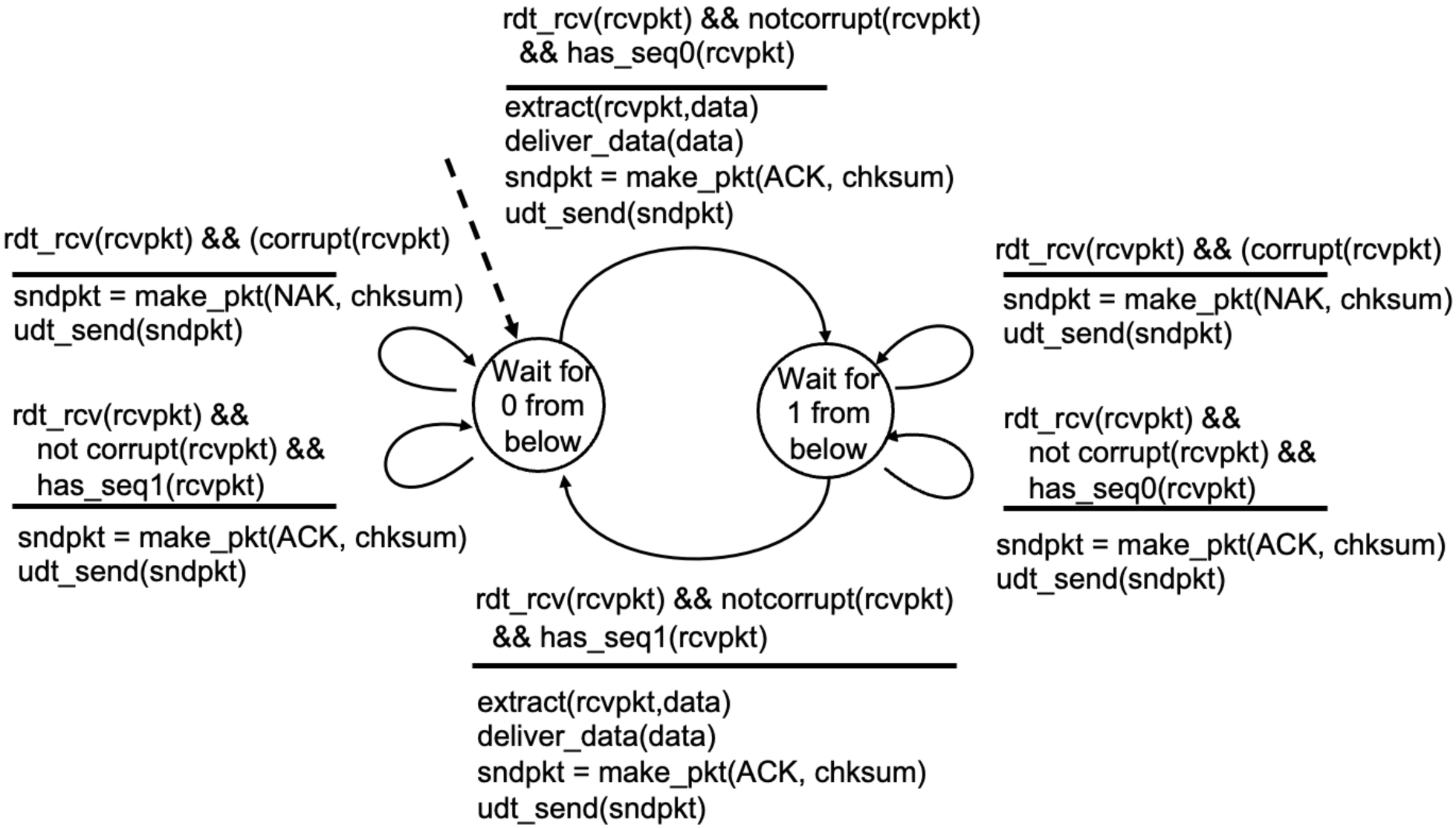
Stop and wait

Sender sends one packet, then waits for receiver response

rdt 2.1: sender, handling garbled ACK/NAK



rdt 2.1: receiver, handling garbled ACK/NAK



rdt 2.1: discussion

- Sender
  - Sequence number (seq. #) is added to packet
  - Two seq. # will suffice. Why?
  - Must check if received ACK/NAK corrupted
  - Twice as many states
    - State must “remember” whether “expected” packet should have seq. # of 0 or 1
- Receiver
  - Must check if received packet is duplicate
    - State indicates whether 0 or 1 is expected packet seq. #
  - Note: receiver can not know if its last ACK/NAK is correctly received at sender

rdt 2.2: a NAK-free protocol

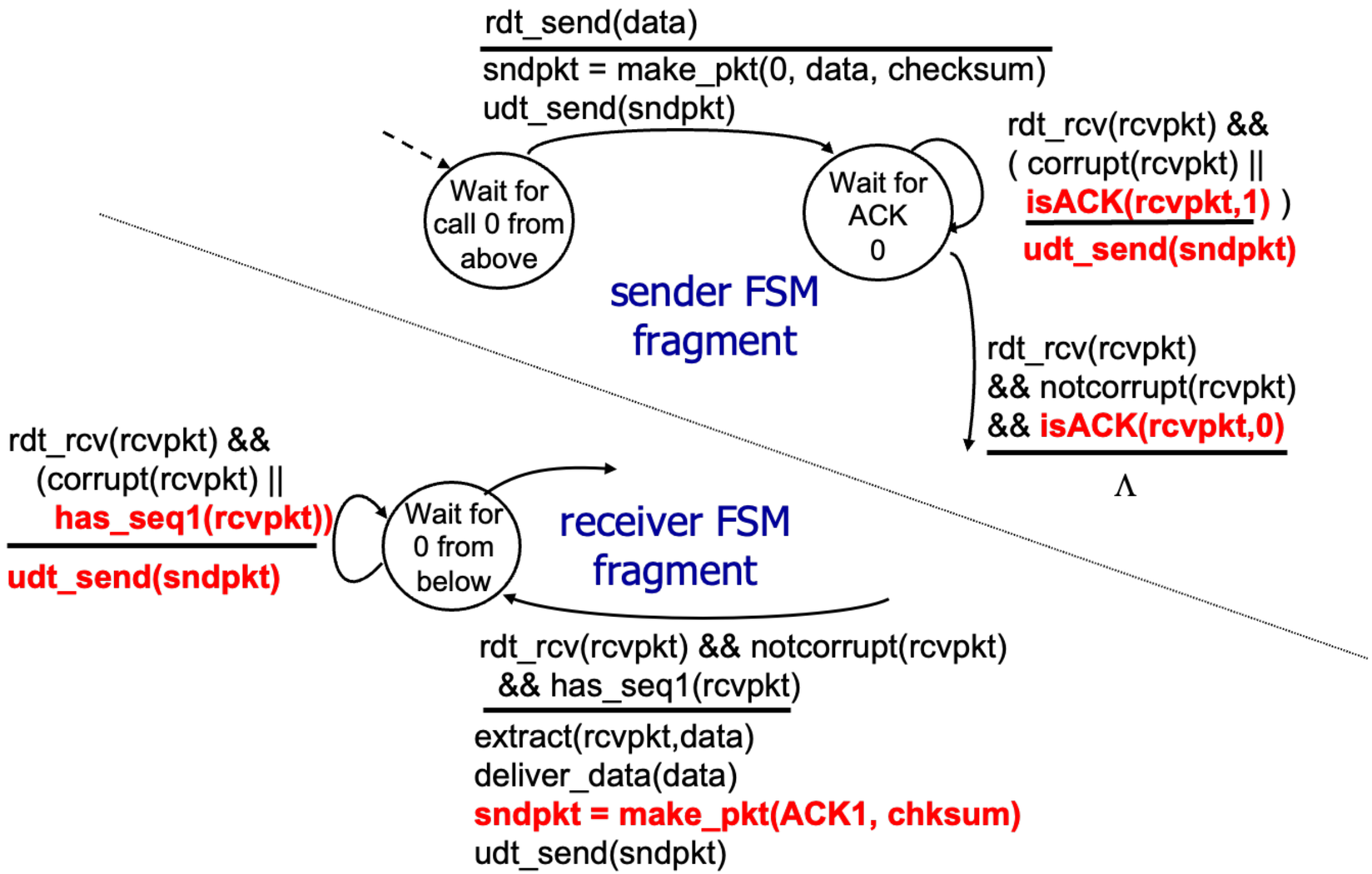
- Same functionality as rdt 2.1, using ACKs only

- Instead of using NAK, receiver sends ACK for the last packet correctly received
  - Receiver must explicitly include seq. # of the packet being ACKed

- Duplicate ACK at sender results in the same action as NAK : retransmit current packet

⇒ As we will see, TCP uses this approach to be NAK-free

rdt 2.2: sender, receiver fragments



rdt 3.0: channels with errors and loss

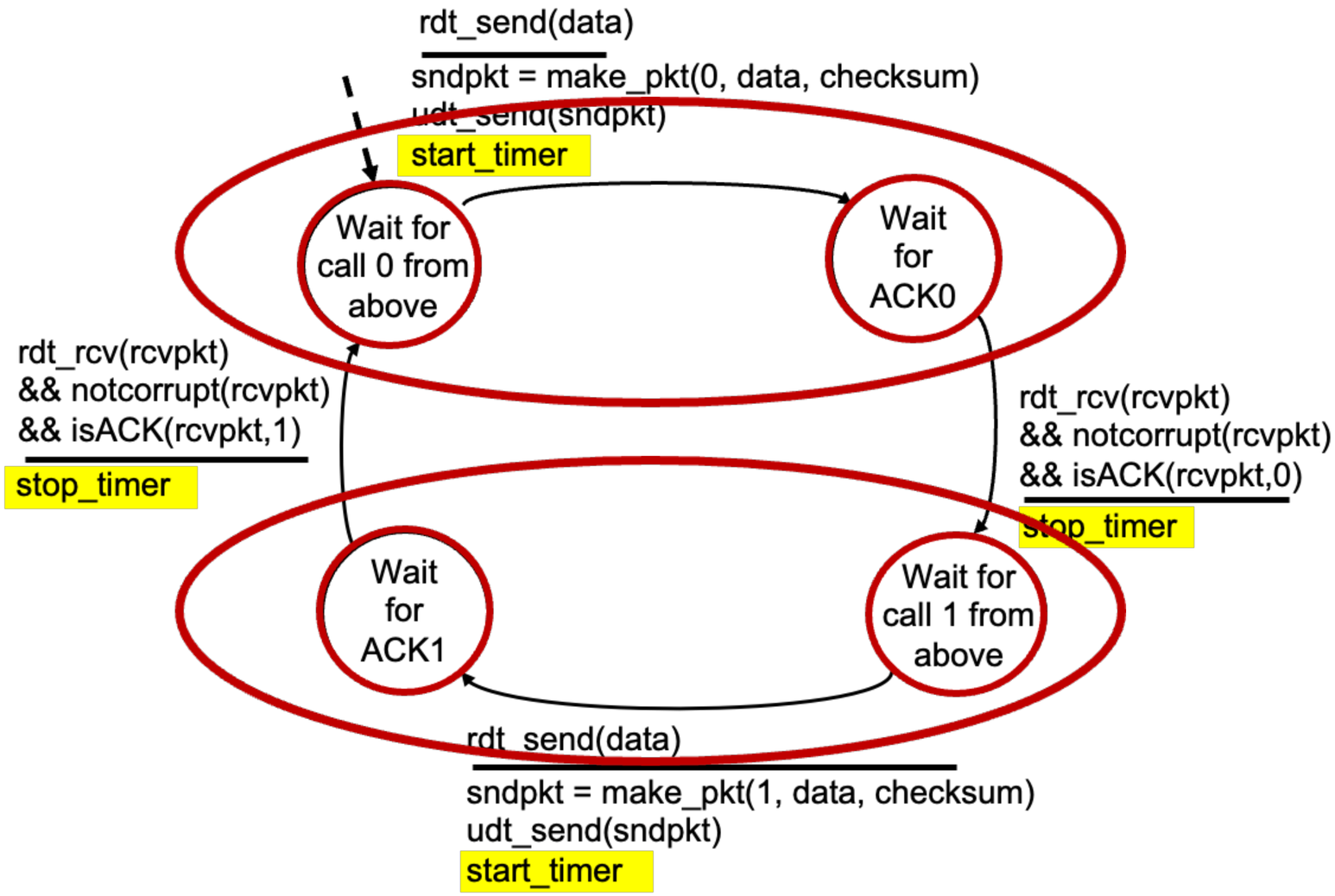
- New channel assumption: underlying channel can also **lose** packet (data, ACKs)
  - Checksum, seq. #s, ACKs, retransmissions will be of help … but not quite enough

Q. How do humans handle lost sender-to-receiver words in conversation?

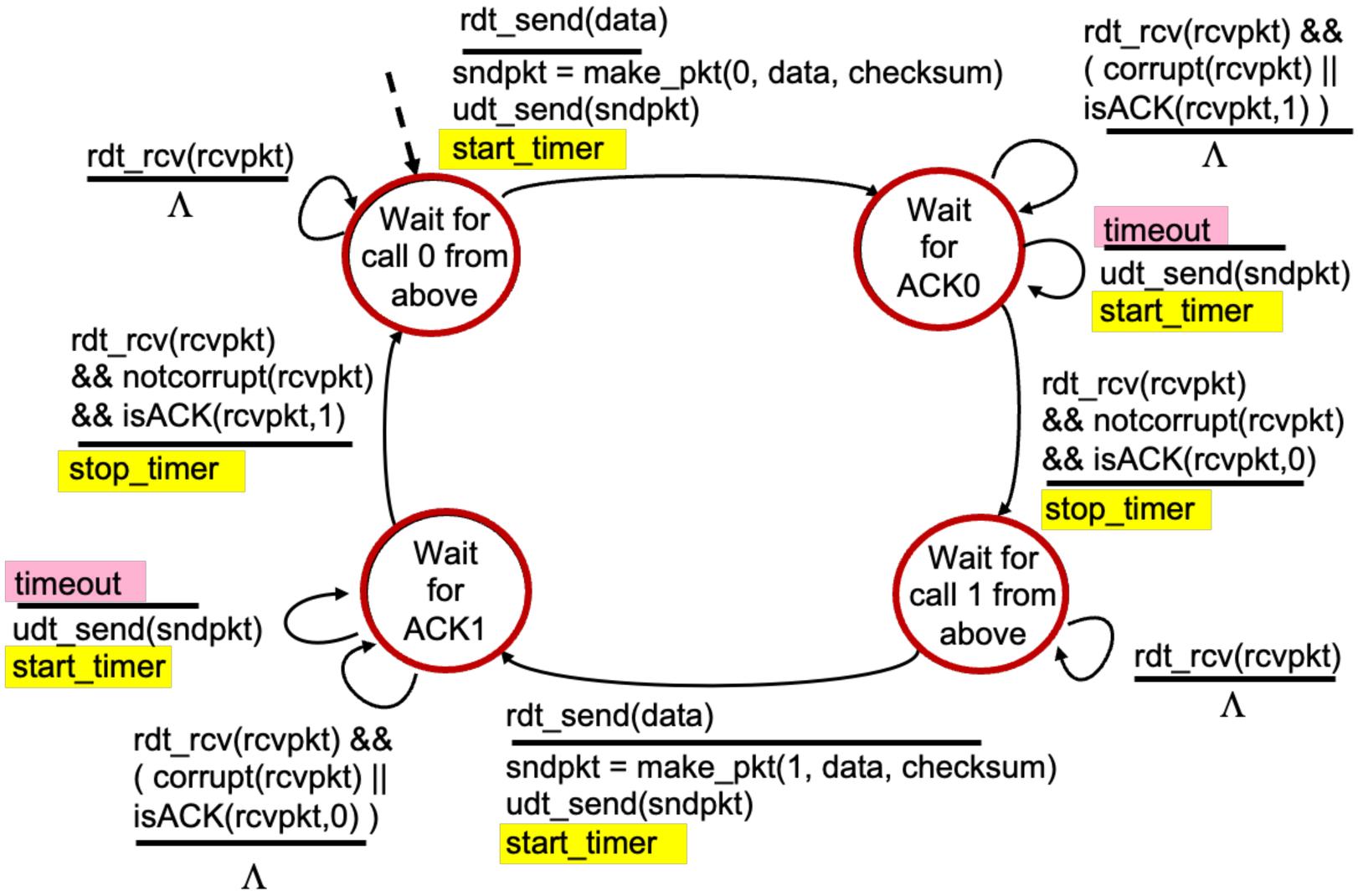
rdt 3.0: channels with errors and loss

- Approach: sender waits “**reasonable**” amount of time for ACK
  - Retransmit if no ACK received in this time
  - If packet (or ACK) is just delayed (not lost):
    - Retransmission will be duplicate, but seq. #s already handles this!
    - Receiver must specify seq. # of packet being ACKed
  - Use countdown timer to interrupt after “reasonable” amount of time

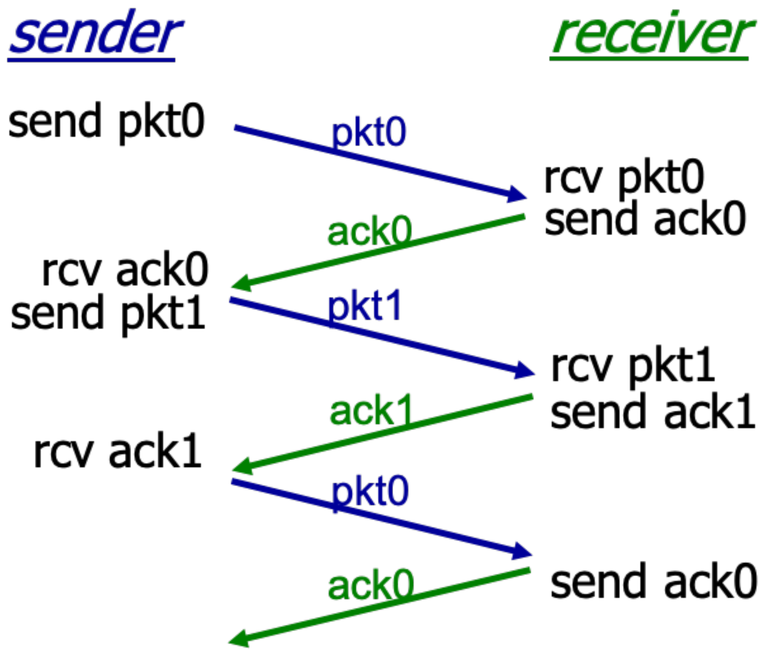
rdt 3.0: sender



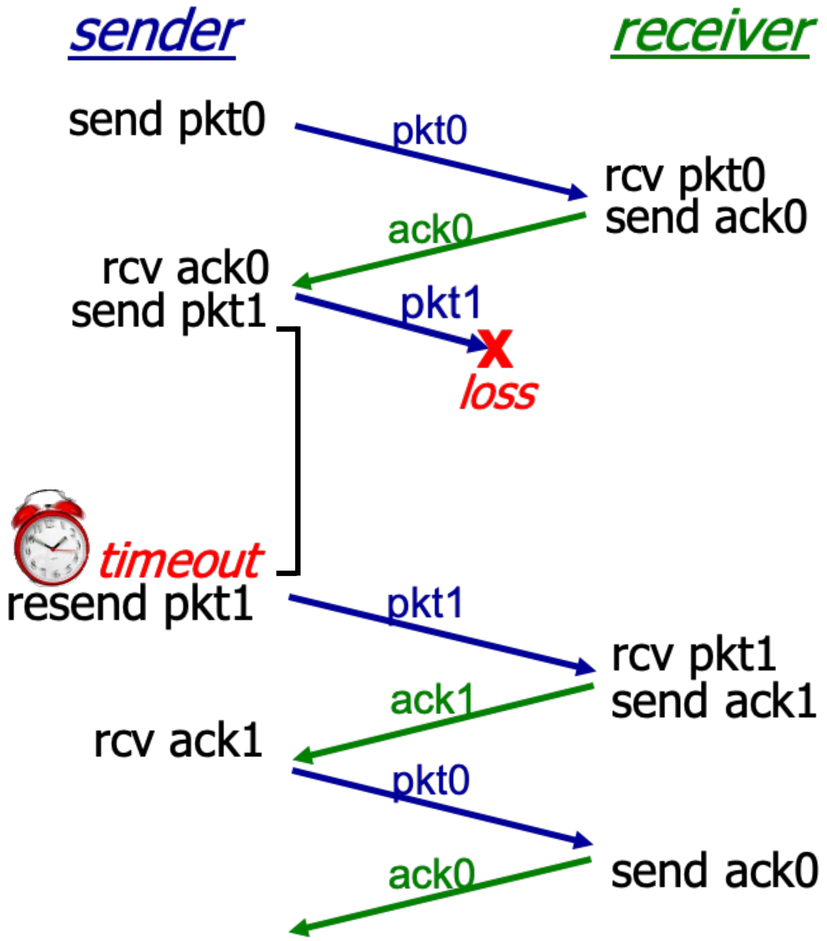
rdt 3.0: sender



rdt 3.0 in action

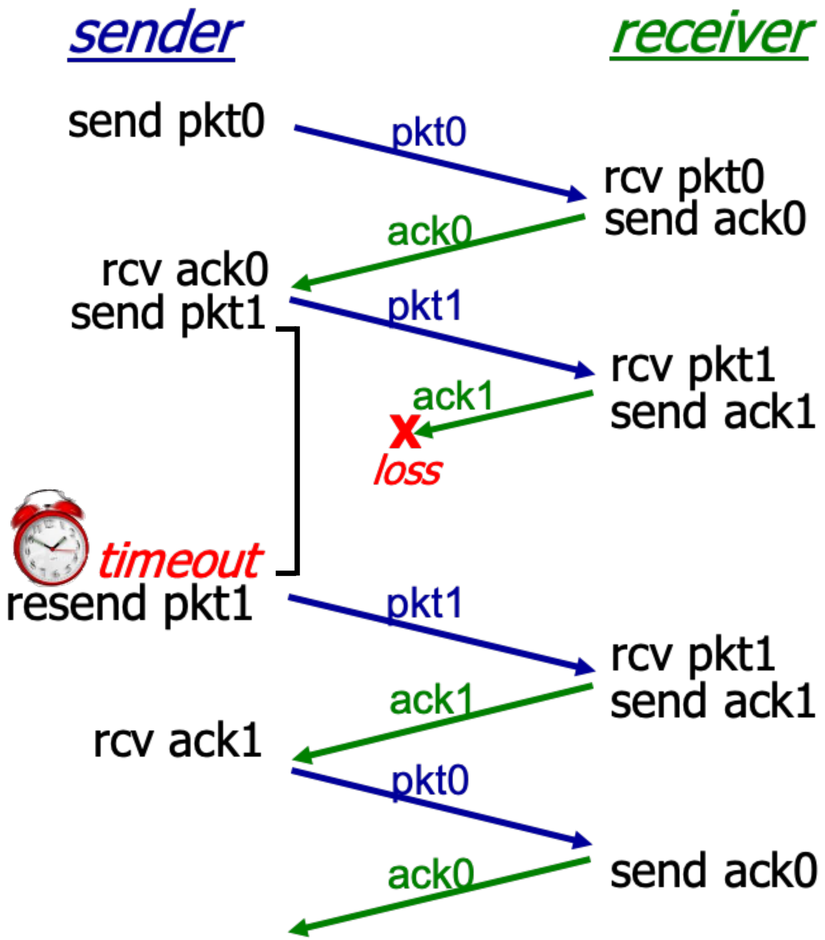


[No loss]

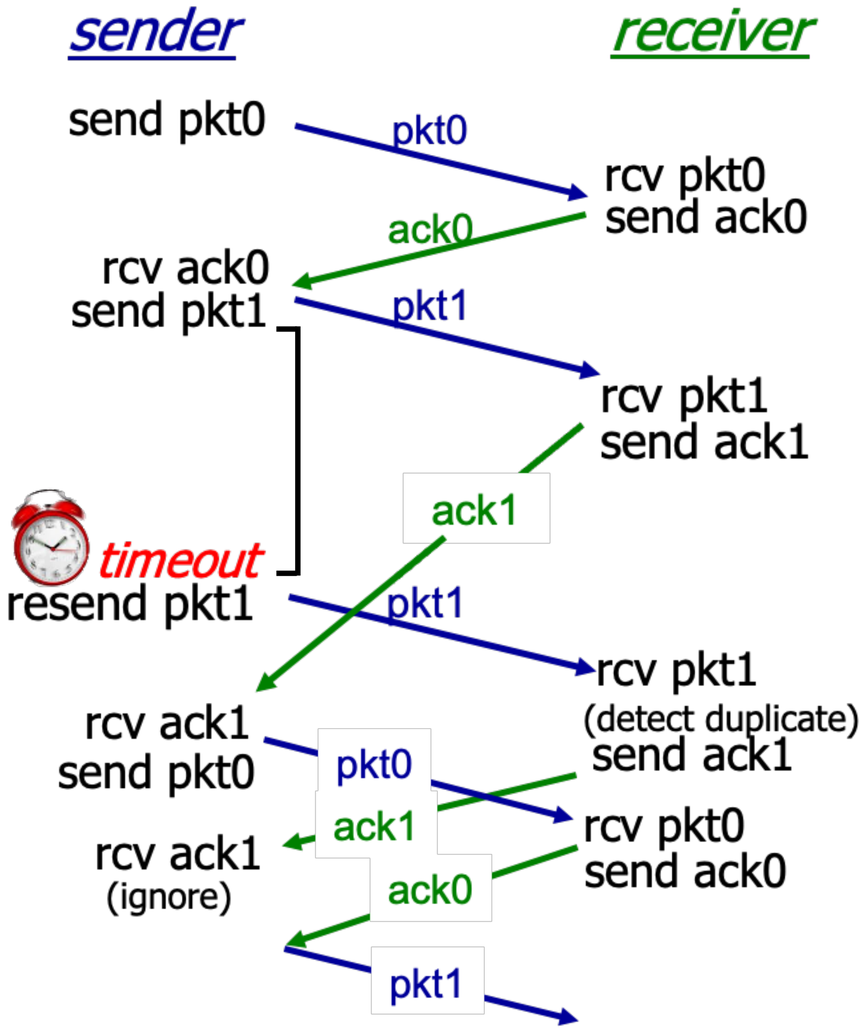


[Packet loss]

rdt 3.0 in action



[ACK loss]



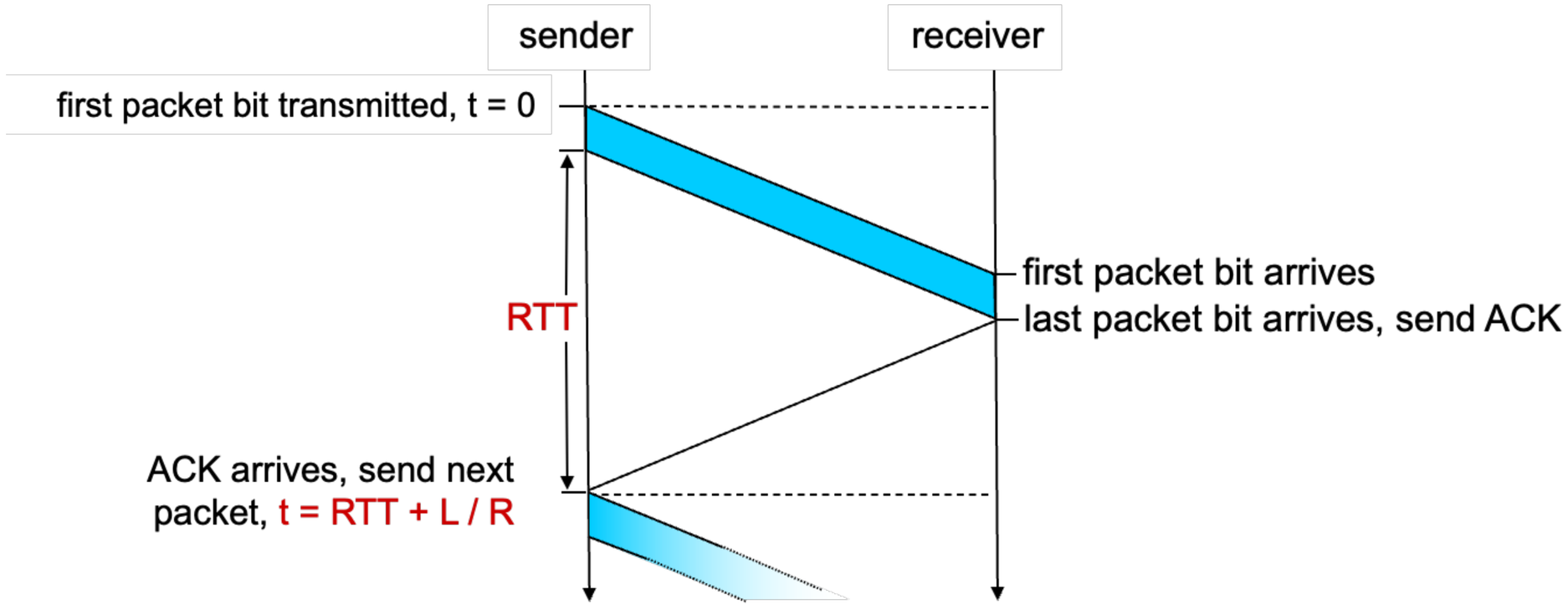
[premature timeout]

Performance of rdt 3.0 (stop-and-wait)

- U_{sender} (utilization) : fraction of time sender busy sending

- e.g., 1Gbps link, 15ms prop. Delay, 8000bit packet
  - D_{trans} (Transmission delay) = $\frac{L}{R}$ = 8000 bits / 10^9 bits/sec = 8msecs

rdt 3.0: stop-and-wait operation

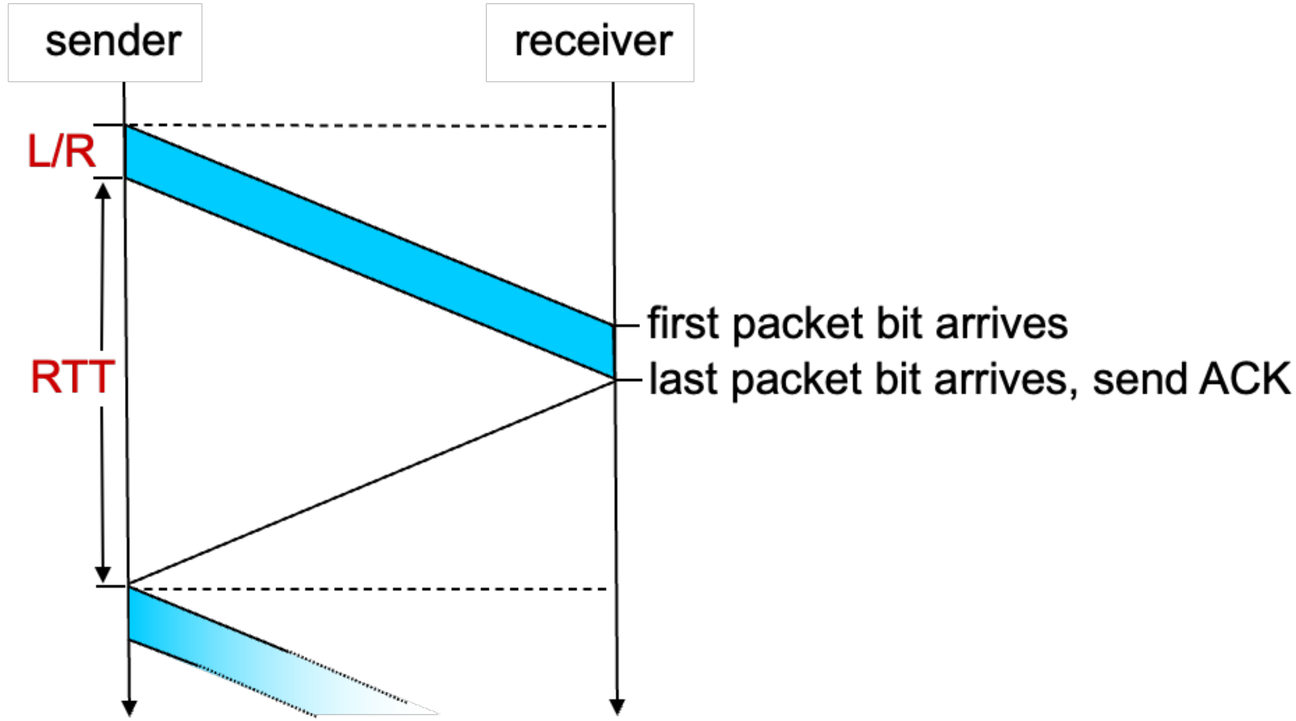


rdt 3.0: stop-and-wait operation

$$U_{sender} = \frac{L / R}{RTT + L / R}$$

$$= \frac{.008}{30.008}$$

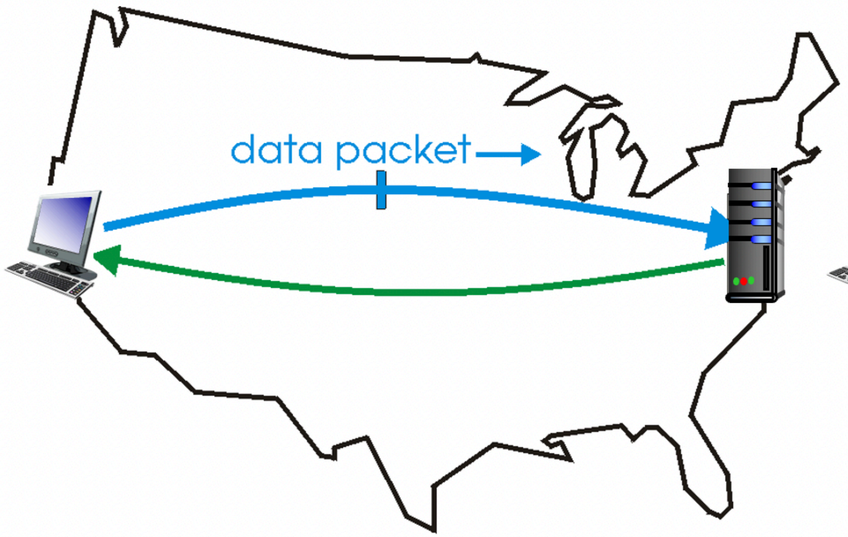
$$= 0.00027$$



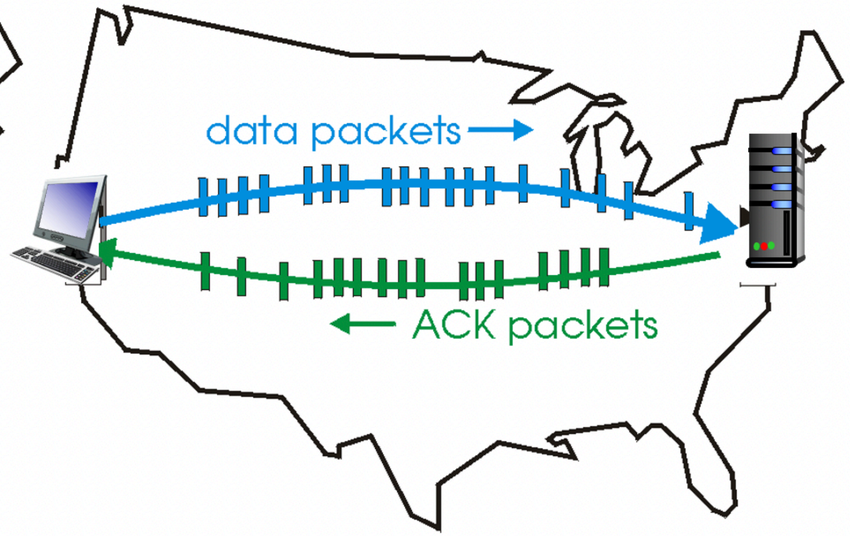
- rdt 3.0 protocol performance stinks!
- Protocol limits performance of underlying infrastructure (channel)

rdt 3.0: pipelined protocols operation

- Pipelining: sender allows multiple, “in-flight” yet-to-be-acknowledged packets
  - Range of sequence numbers must be increased
  - Buffering at sender and/or receiver

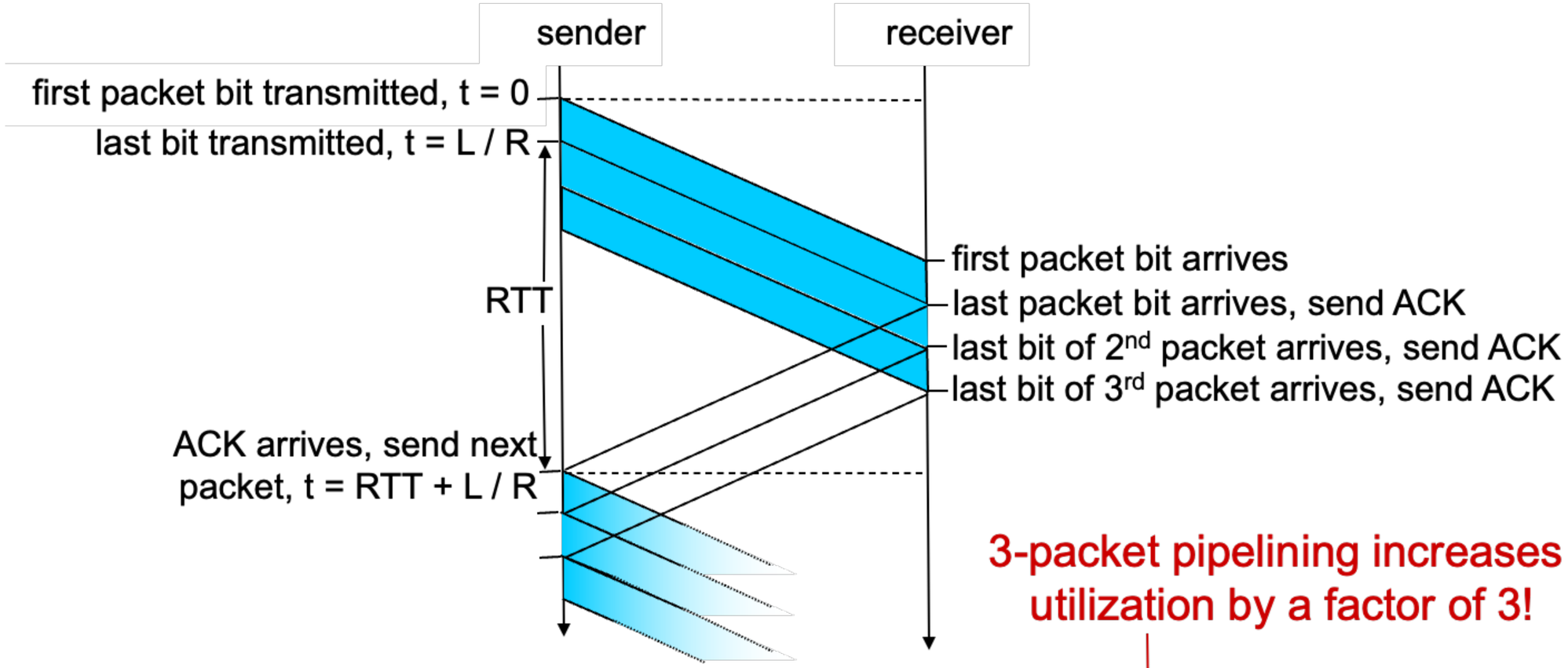


stop-and-wait protocol in operation



Pipelined protocol in operation

Pipelining: increased utilization

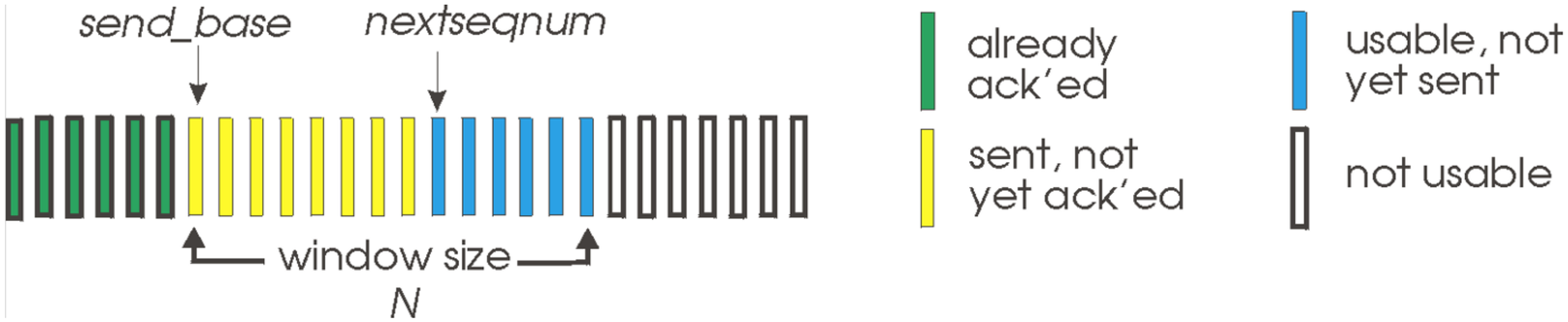


3-packet pipelining increases utilization by a factor of 3!

$$U_{sender} = \frac{3L / R}{RTT + L / R} = \frac{.0024}{30.008} = 0.00081$$

Go-Back-N: sender

- Sender: “window” of up to N, consecutive transmitted but unACKed packets
  - k-bit seq. # in packet header



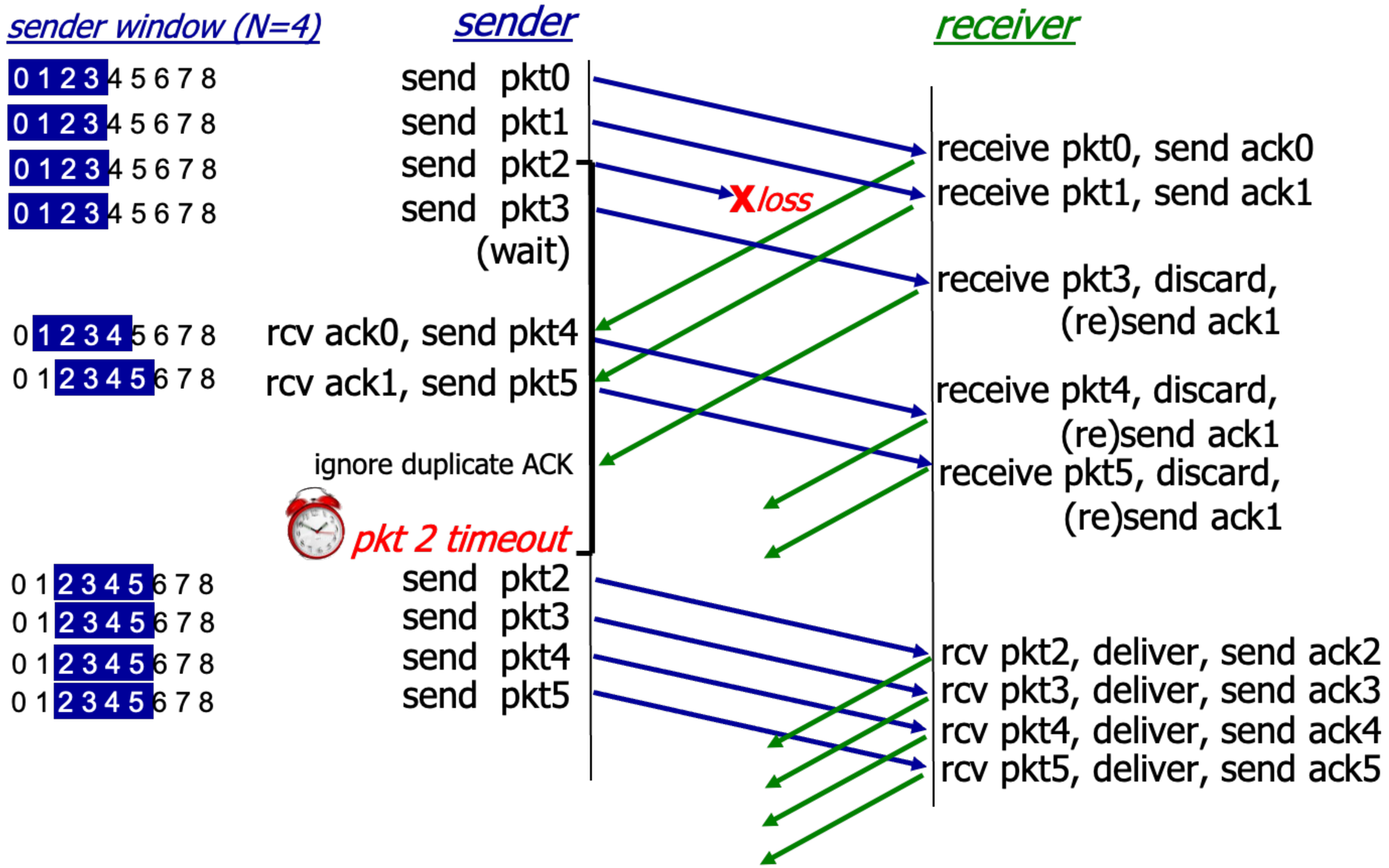
- Cumulative acknowledgement, *ACK*(n): ACKs all packets up to, including seq. # of n
  - On receiving *ACK*(n): move window forward to begin at $n + 1$
- Timer for the oldest in-flight packet
- *timeout*(n): retransmit packet n and all packets with a higher seq. # within the window

Go-Back-N: receiver

- ACK-only: always send ACK for correctly-received packet so far, with the **highest in-order seq. #**
  - May generate duplicate ACKs
  - Need only remember **rcv_base**

- On receipt of out-of-order packet
  - Can discard (don't buffer) or buffer: an implementation decision
  - re-ACK packet with the highest in-order seq. #

Go-Back-N in action



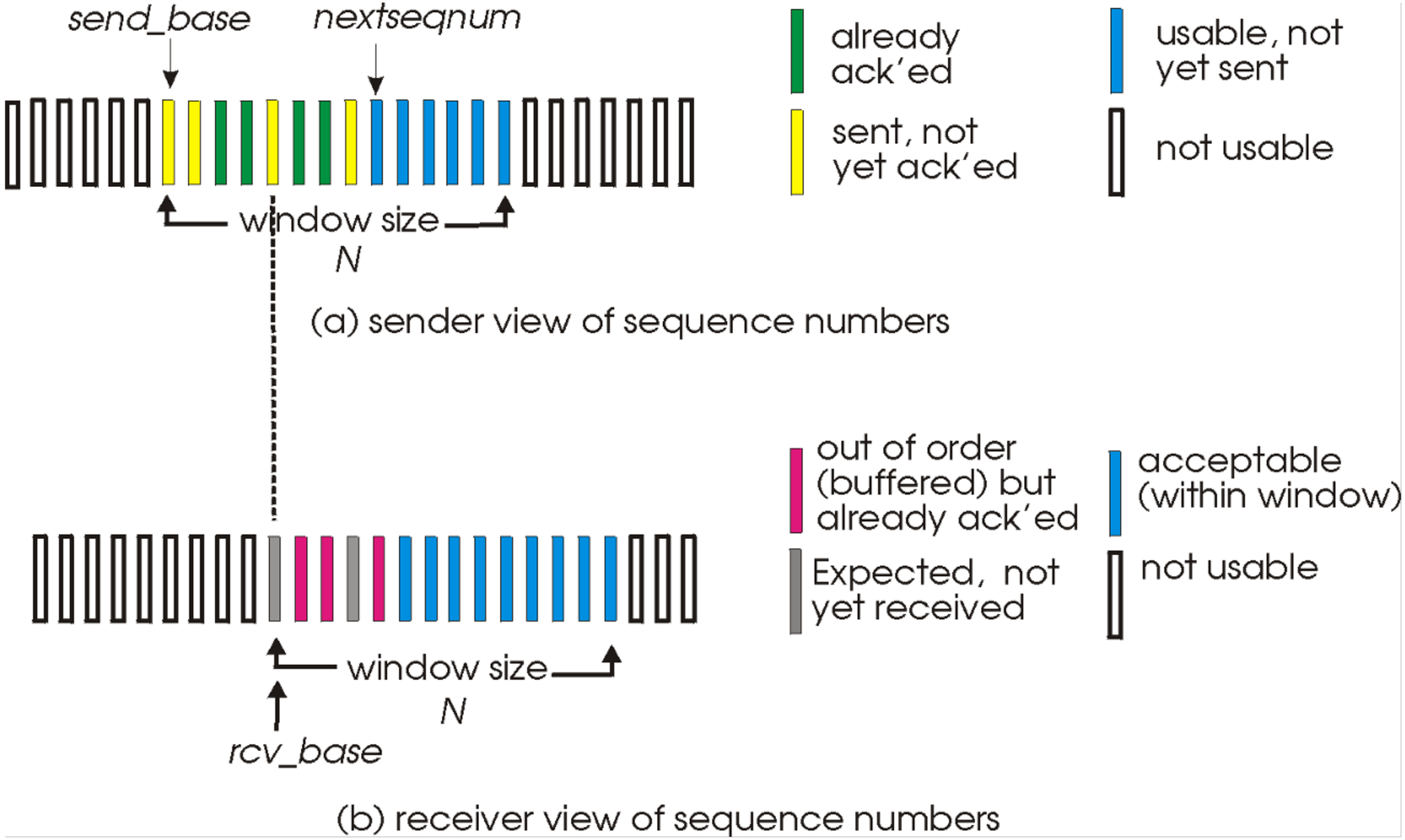
Selective repeat: the approach

- **Pipelining**: multiple packets in flight

- **Receiver individually ACKs** all correctly received packets
  - Buffer packets, as needed, for in-order delivery to upper layer

- Sender:
  - Maintains (conceptually) a timer for each unACKed packet
    - Timeout: retransmits single unACKed packet associated with timeout
  - Maintains (conceptually) “window” over N consecutive seq. #s
    - Limits pipelined, “in-flight” packets to be within this window

Selective repeat: sender, receiver windows



Selective repeat: sender

- Data from above
  - Send packet if next available seq. # is within window

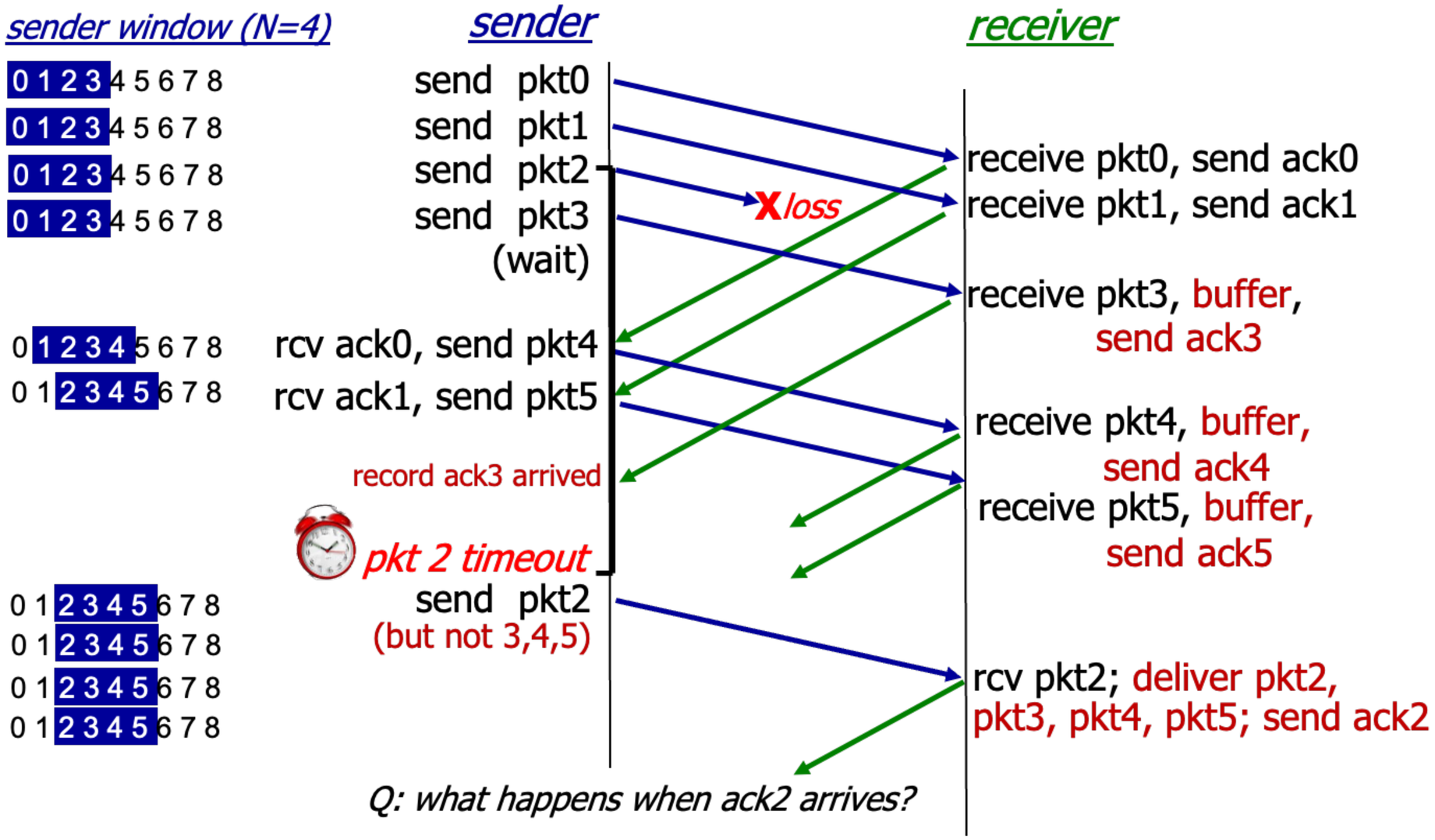
- *timeout*(n)
  - Resend packet n, and restart timer

- ACK(n) in [*sendbase*, *sendbase*+(N-1)]
  - Mark packet n as received
  - Advance window base to next unACKed seq. #, if n is a packet with the smallest unACKed seq. #

Selective repeat: receiver

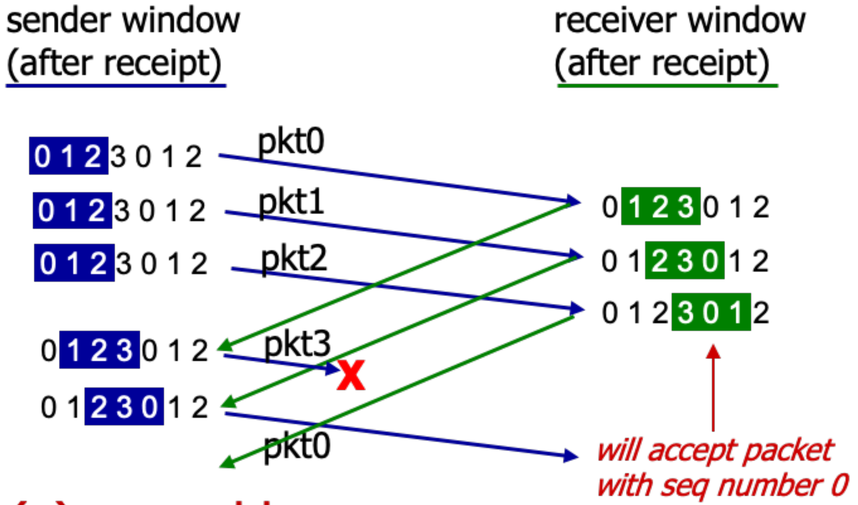
- Packet n in [*rcvbase*, *rcvbase*+(N-1)]
  - Send *ACK*(n)
  - Out-of-order: buffer
  - In-order: deliver (also deliver buffered) in-order packets, and advance window to next not-yet-received packet
- Packet n in [*rcvbase*-N, *rcvbase*-1]
  - *ACK*(n)
- Otherwise
  - ignore

Selective repeat in action

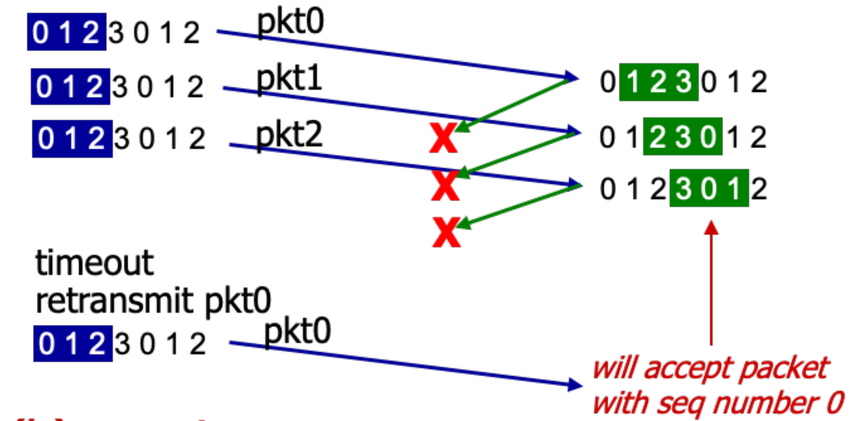


Selective repeat: a dilemma!

- e.g.,
  - Seq. #s: 0, 1, 2, 3 (base 4 counting)
  - Window size = 3



(a) no problem

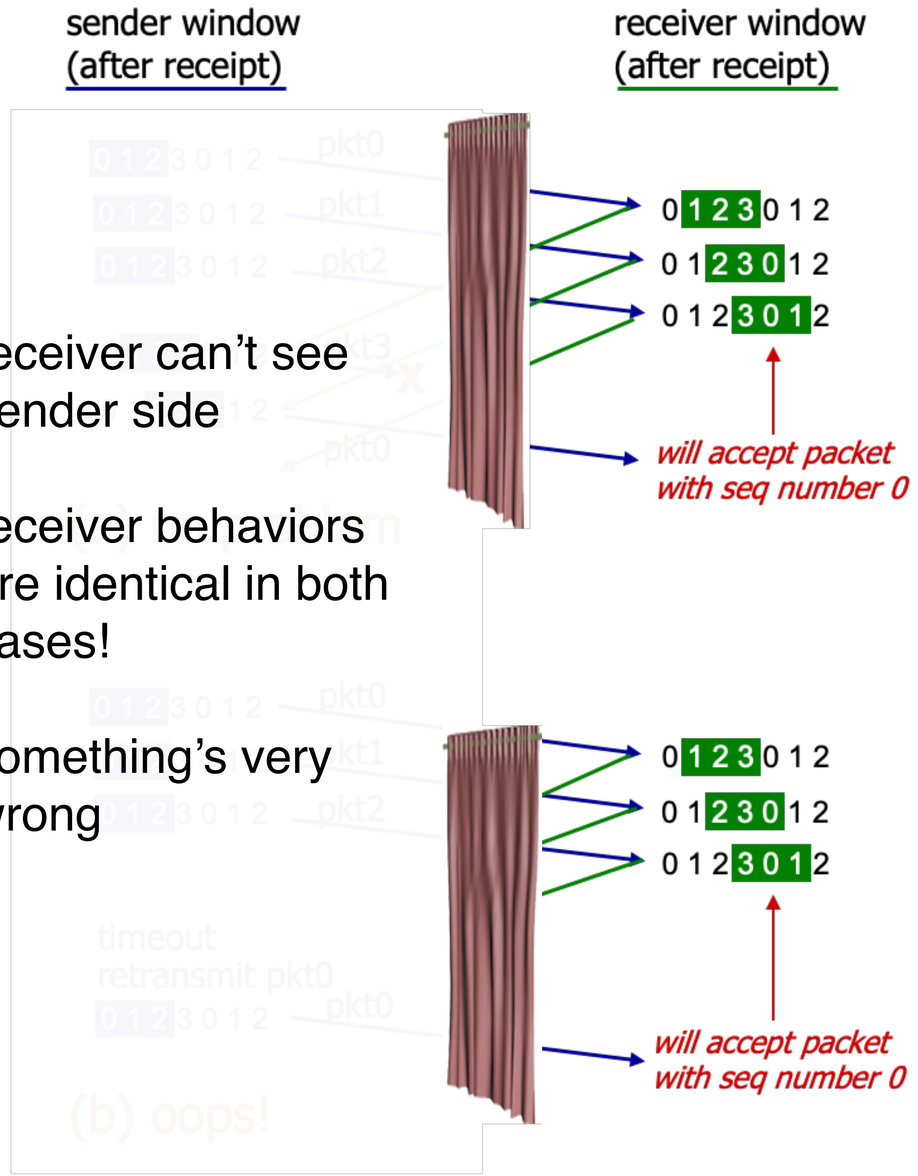


(b) oops!

Selective repeat: a dilemma!

- e.g.,
  - Seq. #s: 0, 1, 2, 3 (base 4 counting)
  - Window size = 3

- What relationship is needed between sequence # size and window size to avoid problem in scenario (b)?

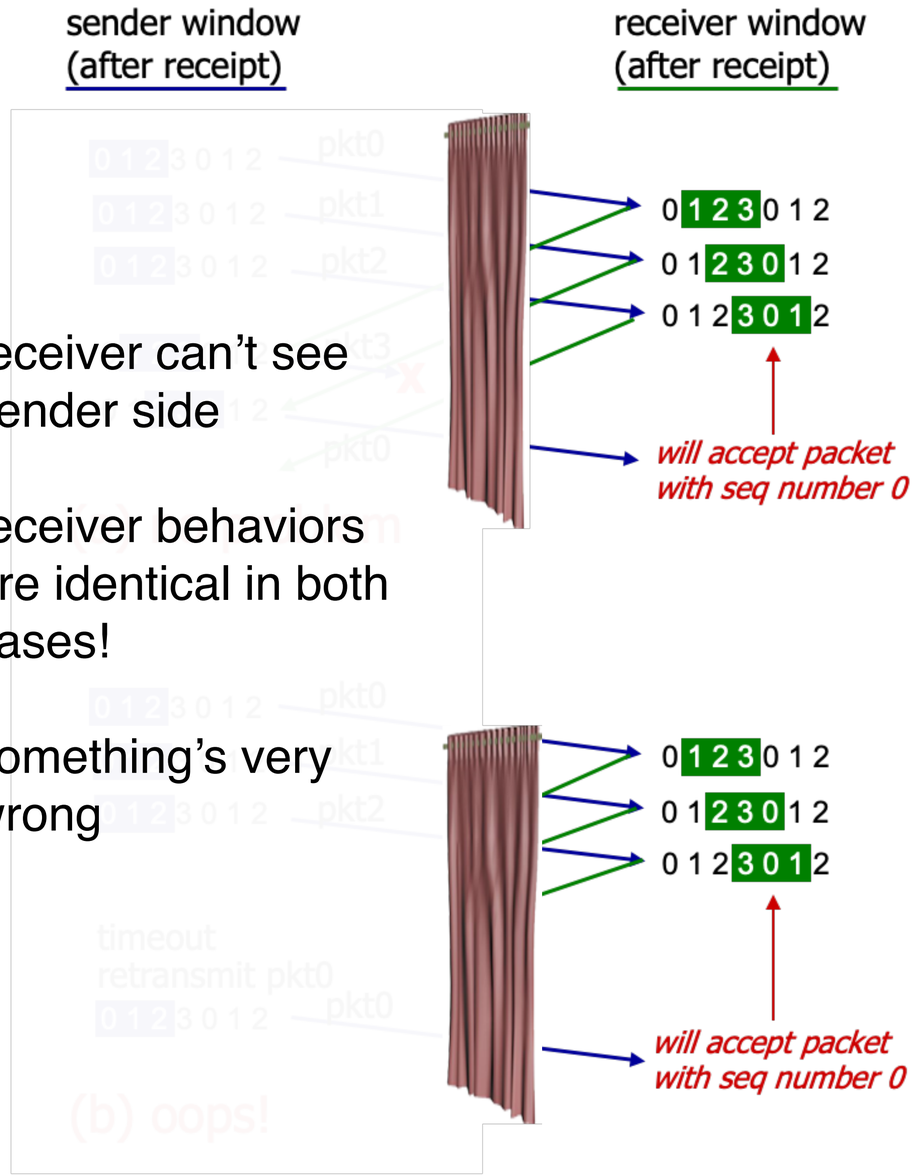


Selective repeat: a dilemma!

- e.g.,
  - Seq. #s: 0, 1, 2, 3 (base 4 counting)
  - Window size = 3

- What relationship is needed between sequence # size and window size to avoid problem in scenario (b)?

  ⇒ SNsize ≥ SWsize + RWsize



Questions?